CSC 1202: Software Development Project

Internship Logging & Evaluation System (ILES)

Project Analysis, Entity-Relationship Diagram & Development Phases

Lecturer: Dr. Peter Khisa Wakholi (Ph.D.)

Email: pwakholi@cit.ac.ug

Year: 2026

1. Project Overview

The Internship Logging and Evaluation System (ILES) is a full-stack web application built using Django (backend) and React (frontend). It is designed to digitize and streamline the internship management process for academic institutions by providing structured workflows for internship placements, weekly activity logging, supervisor reviews, academic evaluations, and institutional reporting.

1.1 System Roles

The system supports four distinct user roles, each with specific responsibilities and access rights:

Role	Responsibilities
Student Intern	Register, submit weekly logs, view evaluations and scores
Workplace Supervisor	Review and approve weekly logs, provide comments
Academic Supervisor	Evaluate students academically, assign weighted scores
Internship Administrator	Manage placements, oversee all workflows, generate reports

1.2 Core Modules

- User & Role Management
- Internship Placement Module
- Weekly Logbook Module
- Supervisor Review Workflow
- Academic Evaluation Module
- Weighted Score Computation
- Dashboards & Reporting

1.3 Technology Stack

Layer	Technology
Backend	Python / Django, Django REST Framework (DRF)
Frontend	React.js, React Router, Recharts
Database	PostgreSQL

Layer	Technology
Authentication	Django Auth, JWT / DRF Token Auth, RBAC
Testing	Pytest (backend), Jest + React Testing Library (frontend)
Deployment	Cloud (Render or equivalent), Environment Variables
Version Control	Git with branching strategy

2. Entity-Relationship Diagram (ERD)

The ERD defines all entities, their attributes, and the relationships between them. The system is built around 7 core entities derived from the project modules.

2.1 Entity Definitions

Entity 1: CustomUser

Field Name	Data Type	Constraints / Notes
id	INT (PK)	Auto-generated primary key
username	VARCHAR(150)	Unique, required
email	VARCHAR(255)	Unique, required
password	VARCHAR(128)	Hashed, required
role	VARCHAR(30)	ENUM: student, workplace_supervisor, academic_supervisor, admin
first_name	VARCHAR(100)	Required
last_name	VARCHAR(100)	Required
phone_number	VARCHAR(20)	Optional
is_active	BOOLEAN	Default: True
date_joined	DATETIME	Auto-set on creation

Entity 2: InternshipPlacement

Field Name	Data Type	Constraints / Notes
id	INT (PK)	Auto-generated primary key
student	FK -> CustomUser	Role must be student; CASCADE on delete
workplace_supervisor	FK -> CustomUser	Role must be workplace_supervisor
academic_supervisor	FK -> CustomUser	Role must be academic_supervisor
organization_name	VARCHAR(255)	Required
organization_address	TEXT	Optional
start_date	DATE	Required; must be before end_date
end_date	DATE	Required; must be after start_date

Field Name	Data Type	Constraints / Notes
status	VARCHAR(20)	ENUM: pending, active, completed, cancelled
created_at	DATETIME	Auto-set on creation

Entity 3: WeeklyLog

Field Name	Data Type	Constraints / Notes
id	INT (PK)	Auto-generated primary key
placement	FK -> InternshipPlacement	Required; CASCADE on delete
student	FK -> CustomUser	Must match placement.student
week_number	INT	Positive integer; unique per placement
week_start_date	DATE	Required
activities_description	TEXT	Required
challenges	TEXT	Optional
learning_outcomes	TEXT	Optional
status	VARCHAR(20)	ENUM: draft, submitted, reviewed, approved, rejected
submitted_at	DATETIME	Nullable; set on submission
created_at	DATETIME	Auto-set on creation
updated_at	DATETIME	Auto-updated on save

Entity 4: SupervisorReview

Field Name	Data Type	Constraints / Notes
id	INT (PK)	Auto-generated primary key
weekly_log	FK -> WeeklyLog	One-to-One; CASCADE on delete
reviewer	FK -> CustomUser	Role: workplace_supervisor
comments	TEXT	Required on review
rating	INT	1-5 scale; optional
action	VARCHAR(20)	ENUM: approved, rejected, revision_requested

Field Name	Data Type	Constraints / Notes
reviewed_at	DATETIME	Auto-set on review

Entity 5: EvaluationCriteria

Field Name	Data Type	Constraints / Notes
id	INT (PK)	Auto-generated primary key
name	VARCHAR(150)	Unique; e.g., Workplace Performance
description	TEXT	Optional
weight_percentage	DECIMAL(5,2)	Must sum to 100 across active criteria
is_active	BOOLEAN	Default: True
created_by	FK -> CustomUser	Admin only
created_at	DATETIME	Auto-set on creation

Entity 6: Evaluation

Field Name	Data Type	Constraints / Notes
id	INT (PK)	Auto-generated primary key
placement	FK -> InternshipPlacement	Required; CASCADE on delete
evaluator	FK -> CustomUser	Role: academic_supervisor
criteria	FK -> EvaluationCriteria	Required
score	DECIMAL(5,2)	0-100 range
remarks	TEXT	Optional
is_submitted	BOOLEAN	Default: False; prevents re-submission
submitted_at	DATETIME	Nullable; set on submission
created_at	DATETIME	Auto-set on creation

Entity 7: ComputedScore

Field Name	Data Type	Constraints / Notes
id	INT (PK)	Auto-generated primary key
placement	FK -> InternshipPlacement	One-to-One; CASCADE on delete
total_score	DECIMAL(5,2)	Auto-computed from Evaluations using weights
grade	VARCHAR(5)	Derived: A, B, C, D, F based on total_score
computed_at	DATETIME	Auto-set on computation
is_final	BOOLEAN	Default: False; set by admin to lock score

2.2 Relationships Summary

Entities	Cardinality	Description
CustomUser -> InternshipPlacement	1 : M	One student can have many placements (but only one active at a time)
CustomUser -> WeeklyLog	1 : M	A student submits many weekly logs
InternshipPlacement -> WeeklyLog	1 : M	One placement has many weekly logs (one per week)
WeeklyLog -> SupervisorReview	1 : 1	Each weekly log has at most one supervisor review
InternshipPlacement -> Evaluation	1 : M	One placement can have evaluations across multiple criteria
EvaluationCriteria -> Evaluation	1 : M	Each criterion can be used in many evaluations
InternshipPlacement -> ComputedScore	1 : 1	One placement has one final computed score
CustomUser -> SupervisorReview	1 : M	One supervisor reviews many weekly logs
CustomUser -> Evaluation	1 : M	One academic supervisor evaluates many placements

2.3 Workflow State Transitions

The system enforces strict state transitions to maintain data integrity throughout the internship lifecycle.

Entity	Valid States	Transition Rules
InternshipPlacement	pending -> active -> completed / cancelled	Admin activates placement; system auto-completes when end_date passes
WeeklyLog	draft -> submitted -> reviewed -> approved / rejected	Student submits; supervisor reviews; approved locks editing
Evaluation	open -> submitted (is_submitted = True)	Once submitted, no further edits allowed (prevent double submission)
ComputedScore	pending -> computed -> final (is_final = True)	Auto-computed when all evaluations submitted; admin locks final score

2.4 Scoring Formula

The weighted score is computed automatically as follows:

```
Total Score = SUM(Evaluation.score *
EvaluationCriteria.weight_percentage / 100)
```

Example scoring breakdown (typical configuration):

Criteria	Weight	Max Contribution
Workplace Performance	40%	40 points
Academic Assessment	30%	30 points
Log Quality & Consistency	30%	30 points
TOTAL	100%	100 points

3. Development Phases

The ILES project is divided into 4 structured development phases aligned with the 12-week course schedule. Each phase builds on the previous and concludes with a defined deliverable.

Phase 1: Foundation & Setup (Weeks 1-3)

This phase establishes the project foundation including the development environment, initial models, database design, and the full ERD. By the end of this phase the team must submit the Proposal, ERD, and Workflow Diagram.

Task	Details
Form project team (3-5 members)	Assign roles: backend lead, frontend lead, DB designer, QA, DevOps
Initialize Git repository	Set up main, develop, and feature branches using Git branching strategy
Initialize Django project	Create virtual environment, install Django, DRF, configure settings.py
Initialize React frontend	Create React app, install React Router, Axios, and Recharts
Define Django models	Implement CustomUser, InternshipPlacement, WeeklyLog, EvaluationCriteria, Evaluation, SupervisorReview, ComputedScore
Configure PostgreSQL	Set up database, apply migrations, verify connections
Draft problem statement & scope	Document what the system does and does not cover
Write complete user stories	Cover all 4 roles; define workflow states clearly
Submit ERD	Final entity diagram with relationships, cardinality, and constraints
Submit workflow state diagram	Draft -> Submitted -> Reviewed -> Approved flow

Deliverable: Proposal & Design Document (ERD + Workflow Diagram) -- Week 3

Phase 2: Core Backend & Authentication (Weeks 4-6)

This phase implements the authentication system, role-based access control, the internship placement module, and the weekly logbook module. All backend APIs for these core features must be functional and secured by the end of this phase.

Task	Details
Implement custom user authentication	Extend AbstractUser, implement role field, admin user creation
Configure JWT/Token authentication	Install djangorestframework-simplejwt, set up token endpoints
Implement DRF permissions	Create IsStudent, IsWorkplaceSupervisor, IsAcademicSupervisor, IsAdmin permission classes
Build role-based login in React	Login form, token storage, user context, role-based routing
Implement React protected routes	Redirect unauthenticated users; restrict routes by role
Build Placement API endpoints	CRUD for InternshipPlacement; validate non-overlapping date ranges
Implement placement date validation	Prevent a student from having two active placements simultaneously
Build Placement UI	Admin can create/manage placements; student views their active placement
Build Weekly Log API endpoints	Create, edit, submit log endpoints; enforce weekly deadline
Implement log state logic	Draft saves at any time; submission locks editing; approval prevents further changes
Build Weekly Log UI	Student form to write and submit logs; status indicators

Deliverable: Working authentication, placement module, and weekly log submission -- internal milestone

Phase 3: Workflow, Evaluation & Dashboards (Weeks 7-10)

This phase implements the supervisor review workflow with audit trails, the academic evaluation and weighted score computation engine, and all role-specific dashboards. The Midterm Prototype Presentation at Week 8 is a key milestone within this phase.

Task	Details
Implement supervisor review API	Allow workplace supervisor to approve, reject, or request revision on weekly logs
Enforce valid state transitions	Prevent invalid transitions (e.g., draft -> approved without submission)
Implement audit trail / status history	Log each state change with timestamp, actor, and previous state (Django Signals)
Build supervisor review UI	Review interface with comment box, action buttons, and log history
Midterm Prototype Presentation	Demo: authentication, weekly workflow, supervisor review, basic dashboards
Implement EvaluationCriteria management	Admin can create, weight, and activate/deactivate criteria
Build Evaluation API endpoints	Academic supervisor submits scores per criterion; prevent duplicate submissions
Implement weighted score computation	Auto-calculate ComputedScore on all evaluations submitted; store derived grade
Build student progress dashboard	Show log submission status, review results, and computed score
Build supervisor pending reviews dashboard	List logs awaiting review; filter by week and status
Build admin statistics dashboard	Aggregated view: total interns, placements, completion rates, score distributions
Implement React charts	Use Recharts for score distribution, log completion progress

Deliverable: Midterm Prototype (Week 8) -- Core system functional including auth, workflows, and basic dashboards

Phase 4: Testing, Deployment & Technical Defense (Weeks 11-12)

The final phase focuses on ensuring system quality through testing, preparing the application for production deployment, writing all required documentation, and defending the technical design decisions before the examination panel.

Task	Details
Write backend unit tests (min 10)	Use Pytest: test model methods, API endpoints, permissions, state transitions
Write frontend tests (min 5)	Use Jest + React Testing Library: test components, routing, forms
Generate test coverage report	Use pytest-cov; ensure critical modules have minimum 70% coverage
Fix detected bugs	Address all bugs found during testing before deployment
Configure production Django settings	DEBUG=False, allowed hosts, secure cookies, static files, env variables
Deploy to cloud (Render)	Deploy Django API and React frontend; configure PostgreSQL database on cloud
Final system cleanup & code review	Remove debug code, clean comments, standardize naming conventions
Write user manual	Step-by-step guide for each role on using the system
Write technical documentation	API endpoints, DB schema, deployment steps, architecture overview
Prepare for technical defense	Be ready to explain ERD, workflow, scoring formula, and code walkthrough

Deliverable: Final Deployed System + User Manual + Technical Documentation + Live Defense -- Week 12

4. Assessment & Grading Overview

Assessment Component	Weight	Relevant Phase
Proposal & Requirements	10%	Phase 1
ERD & Workflow Design	10%	Phase 1
Backend Implementation	15%	Phase 2 & 3
Frontend Implementation	10%	Phase 2 & 3
Workflow & Business Logic	10%	Phase 3
Testing & Code Quality	5%	Phase 4
Programming Quizzes	10%	All Phases
Practical Coding Exam	40%	All Phases
Final Presentation & Technical Defense	10%	Phase 4

Note: Continuous Assessment makes up 40% of the final grade, the Practical Coding Exam accounts for 40%, Programming Quizzes for 10%, and the Final Presentation & Defense for the remaining 10%.

5. Project Deliverables & Timeline

Phase	Week	Deliverable	Description
Phase 1	Week 3	Proposal & Design	ERD, workflow diagram, problem statement, user stories
Phase 2	Week 6	Core Backend Milestone	Auth, placements, and weekly logs fully functional (internal)
Phase 3	Week 8	Midterm Prototype	Demo: core system, workflows, supervisor review, dashboards
Phase 4	Week 12	Final System	Deployed system, user manual, technical docs, live defense